VISION AI

Plataforma inteligente de análisis visual

Detección de objetos · Segmentación · Reconocimiento de patrones

PROPUESTA DE SISTEMA SIMULADO FUNCIONAL PARA EMPRESAS

Documento guía para diseño, desarrollo, pruebas y demostración

Control del documento

Campo	Definición
Nombre del proyecto	VISION AI – Plataforma inteligente de análisis visual
Tipo	Sistema web empresarial; prototipo funcional con datos e inferencia simulada
Versión	1.0
Objetivo	Diseñar y construir una solución demostrable que explique y simule detección, segmentación y reconocimiento de patrones.
Alcance tecnológico	Frontend React + TypeScript; motor de simulación local; arquitectura preparada para conectar un servicio real de IA.
Usuarios	Administrador, operador de análisis, supervisor y auditor.

1. Resumen ejecutivo

VISION AI es una plataforma web empresarial que convierte imágenes o fotogramas de video en resultados visuales interpretables. El usuario carga una imagen de prueba o selecciona una escena de demostración, ejecuta un análisis y visualiza objetos detectados, máscaras de segmentación, patrones identificados, métricas y alertas. El sistema se plantea como una simulación funcional: los flujos, formularios, estados, gráficos, filtros y resultados de demostración deben operar realmente en el navegador. La arquitectura separa la interfaz del motor de análisis para sustituir posteriormente la simulación por modelos de visión computacional desplegados en un servicio backend.

Importante: una simulación con resultados predefinidos o reglas de demostración no debe presentarse como un modelo entrenado ni como una inferencia real. La interfaz identificará claramente el modo DEMO. La integración de un modelo real será una fase posterior y medible.

2. Objetivos de aprendizaje y fundamentos

2.1 Detección de objetos

La detección de objetos consiste en localizar y clasificar una o varias instancias dentro de una imagen o fotograma. El resultado típico contiene una etiqueta de clase (por ejemplo, persona, vehículo o casco), una puntuación de confianza y una caja delimitadora (bounding box) definida por coordenadas x, y, ancho y alto.

- Flujo conceptual: captura/imagen → preprocesamiento → modelo detector → filtrado por umbral y supresión de duplicados (NMS) → cajas, clases y confianza.
- Métricas habituales: precisión, recall, IoU (intersección sobre unión), mAP y latencia por imagen.
- Ejemplo empresarial: detectar personas y vehículos en una zona de acceso, o identificar productos en una línea de producción.

2.2 Segmentación

La segmentación asigna una clase o identidad a los píxeles de una imagen. A diferencia de la detección, que delimita objetos mediante rectángulos, la segmentación describe su contorno o región con mayor precisión.

Tipo	Qué produce	Uso típico
Semántica	Máscara por clase; todos los píxeles de una clase comparten etiqueta.	Separar carretera, piso, producto o fondo.
Instancia	Máscara individual para cada objeto detectado.	Contar productos o personas sin fusionar instancias.
Panóptica	Combina regiones semánticas y objetos individuales.	Comprensión integral de una escena.

2.3 Reconocimiento de patrones

Es el proceso de identificar regularidades, características o comportamientos en datos visuales y asignarlos a categorías o eventos. Puede utilizar descriptores, embeddings, clasificadores, redes neuronales o reglas temporales. En este proyecto se simularán patrones como uso de casco, objeto fuera de zona, producto defectuoso y congestión.

- Clasificación: asignar una categoría a una imagen o región.
- Anomalías: señalar desviaciones respecto de un patrón esperado.
- Seguimiento temporal: asociar detecciones entre fotogramas para estimar trayectorias o permanencia.
- La confianza es una estimación del modelo, no una garantía de certeza; las decisiones críticas requieren validación humana.

3. Aplicaciones empresariales principales

Sector	Aplicación	Resultado operativo
Seguridad y prevención	Conteo de personas, EPP/casco, zonas restringidas.	Alertas y revisión de incidentes.
Manufactura	Inspección visual, defectos, piezas ausentes.	Control de calidad y trazabilidad.
Retail y almacenes	Conteo de productos, ocupación, inventario visual.	Reposición y análisis de flujo.
Transporte y logística	Conteo/clasificación de vehículos y lectura de zonas.	Flujo de patio y tiempos de atención.
Agricultura	Detección de frutos, hojas afectadas o maleza.	Estimación y monitoreo de cultivos.
Salud e investigación	Análisis asistido de imágenes bajo protocolos especializados.	Soporte a revisión; no diagnóstico autónomo.

4. Alcance del sistema simulado

4.1 Funcionalidades incluidas

- Inicio de sesión demostrativo y perfiles/roles de interfaz.
- Dashboard con KPI: análisis ejecutados, objetos detectados, alertas, confianza media y tiempo de procesamiento.
- Centro de análisis: cargar imagen JPG/PNG, elegir escenario demo y ejecutar análisis.
- Visor visual con bounding boxes, etiquetas, colores por clase, máscaras superpuestas y leyenda.
- Panel de resultados con lista de detecciones, confianza, clase, coordenadas y filtros.
- Reconocimiento de patrones y reglas de alerta configurables.
- Historial de análisis con búsqueda, filtros, detalle y exportación CSV.
- Gestión de escenarios, clases, umbrales, zonas y reglas de demostración.
- Notificaciones visuales en tiempo real dentro de la sesión y registro de eventos.
- Estados de carga, progreso, éxito, advertencia, error y ausencia de datos; animaciones discretas.

4.2 Fuera del alcance de la primera demo

- Vigilancia continua 24/7, grabación o almacenamiento masivo de video.
- Identificación biométrica de personas o reconocimiento facial.
- Decisiones automáticas disciplinarias, médicas o de seguridad sin supervisión.
- Precisión de producción o cumplimiento de SLA sin pruebas con datos representativos.

5. Arquitectura de software propuesta

Se recomienda una arquitectura modular por capas y orientada a casos de uso. La interfaz no debe contener la lógica del modelo: consume contratos de servicio. Esto facilita mantenimiento, pruebas y reemplazo del motor simulado.

Capa	Responsabilidad	Componentes sugeridos
Presentación	Navegación, componentes, estados y visualización.	React, TypeScript, Tailwind/shadcn, Recharts, Lucide.
Aplicación	Orquestar casos de uso y validar entradas.	AnalyzeImage, GetDashboard, SearchHistory, ManageRules.
Dominio	Entidades, reglas y contratos independientes de UI.	Analysis, Detection, SegmentationMask, Pattern, Alert, Scenario.
Infraestructura	Persistencia, archivos, exportaciones y adaptadores.	Repositorio local DEMO; luego API REST, PostgreSQL y almacenamiento de objetos.
Servicio IA	Preprocesamiento, inferencia y postprocesamiento.	DemoVisionAdapter; futuro Python + FastAPI + OpenCV + modelo YOLO/segmentación.

5.1 Diagrama lógico

USUARIO → REACT / PRESENTACIÓN → CASOS DE USO → DOMINIO

↓

PUERTOS / INTERFACES

↓

ADAPTADOR DEMO | API REAL DE VISIÓN

↓

RESULTADOS → REPOSITORIO → DASHBOARD / HISTORIAL

5.2 Estructura de carpetas sugerida

```
src/
  app/
    router/                # rutas y protección por rol
    providers/             # tema, sesión, query client
    layout/                # shell, sidebar, header, breadcrumbs
  modules/
    dashboard/
      pages/ components/ hooks/ services/
    analysis/
      pages/ components/ hooks/ services/ validators/
    detections/
      components/ services/
    segmentation/
      components/ services/
    patterns/
      pages/ components/ rules/
    alerts/
      pages/ components/ services/
    history/
      pages/ components/ export/
    scenarios/
      pages/ components/ data/
    settings/
      pages/ components/
  core/
    domain/                # entidades, enums, reglas
    application/           # casos de uso
    ports/                 # interfaces
    contracts/             # DTO y esquemas
  infrastructure/
    vision/
      DemoVisionAdapter.ts
      ApiVisionAdapter.ts
    persistence/
      LocalAnalysisRepository.ts
    export/
  shared/
```

```
    ui/ hooks/ utils/ types/
  styles/
  tests/
    unit/ integration/ e2e/
server/                    # fase de integración real
  api/
  vision/
    preprocessing/
    detection/
    segmentation/
    patterns/
  tests/
```

5.3 Navegación sin saturar la pantalla

Evitar un acordeón interminable o decenas de pestañas. Usar un menú lateral con módulos principales, subrutas dentro del área de trabajo, breadcrumbs y acciones contextuales. El panel de análisis concentra el visor y un panel lateral de resultados; las configuraciones avanzadas se mantienen en páginas independientes.

Menú principal	Submódulos / páginas
Resumen	Dashboard, actividad reciente, indicadores.
Análisis visual	Nuevo análisis, galería de escenarios, resultados.
Visión IA	Detecciones, segmentación, patrones.
Operaciones	Alertas, historial, exportaciones.
Administración	Escenarios, clases, reglas, usuarios, configuración.

6. Diseño UX/UI empresarial

- Estética: enterprise moderna, fondo gris azulado muy claro, superficies blancas, azul profundo para navegación y cyan para acciones/estados activos.
- Tipografía: Inter o Manrope; jerarquía clara, tablas compactas legibles y espacios consistentes.
- Dashboard: tarjetas KPI, gráfico de actividad, distribución por clase, alertas recientes y análisis recientes.
- Área de análisis: visor central amplio; panel lateral con pestañas Resultado / Detecciones / Máscaras / Patrones; barra superior con escenario, modo y botón Analizar.
- Animaciones: entrada suave de tarjetas, progreso de análisis, aparición escalonada de detecciones, pulso moderado en alertas activas y transición de filtros. Respetar prefers-reduced-motion.
- Accesibilidad: contraste suficiente, estados no comunicados solo por color, navegación por teclado y textos alternativos.

6.1 Estados y notificaciones

Estado	Comportamiento
Preparado	Imagen/escenario seleccionado; botón Analizar habilitado.
Procesando	Barra de progreso, etapa actual y botón bloqueado para evitar doble envío.
Completado	Mostrar cajas/máscaras, métricas y toast de éxito.
Advertencia	Confianza baja, imagen no compatible o regla activada; solicitar revisión.
Error	Mensaje comprensible, conservar entrada y ofrecer reintento.
Vacío	Guía para cargar imagen o seleccionar escenario.

7. Flujos funcionales y reglas de negocio

7.1 Flujo de análisis

1. El operador abre Análisis visual y selecciona una imagen o escenario de demostración.
2. El sistema valida formato, tamaño y existencia de entrada.
3. El operador selecciona perfil de análisis: detección, segmentación o patrones; puede combinar los tres.
4. Al pulsar Analizar, se crea un registro con estado EN_PROCESO y se muestra progreso.
5. El adaptador DemoVisionAdapter genera resultados deterministas del escenario seleccionado; la UI dibuja overlays sobre la imagen.
6. Se aplican umbrales y reglas; se generan alertas si corresponde.
7. Se guarda el resumen en el historial y se actualizan los KPI del dashboard.
8. El usuario revisa, confirma observaciones y exporta el reporte.

7.2 Reglas esenciales

- Cada análisis tiene identificador único, fecha, usuario, escenario, modo, estado y duración.
- La confianza se expresa entre 0 y 1 internamente y como porcentaje en pantalla.
- El umbral de confianza es configurable; detecciones inferiores se ocultan o marcan como baja confianza, sin borrarlas del registro bruto.
- Las cajas y máscaras deben usar coordenadas relativas normalizadas (0–1) para adaptarse al tamaño del visor.
- Una alerta tiene severidad, regla que la originó, fecha, estado (nueva, revisada, cerrada) y responsable.
- No se debe afirmar que una persona incumple una norma solo por una detección incierta; mostrar evidencia y permitir revisión.
- El modo demo debe permanecer visible en cabecera y reportes.

8. Modelo de datos

Entidad	Campos principales
Analysis	id, createdAt, createdBy, sourceName, scenarioId, mode, status, durationMs, averageConfidence
Detection	id, analysisId, classId, label, confidence, x, y, width, height, trackId?
Segmentation	id, analysisId, classId, maskRef o maskData, areaRatio, confidence
Pattern	id, analysisId, patternType, label, confidence, evidence, status
Alert	id, analysisId, ruleId, severity, message, status, createdAt, reviewedBy
Class	id, name, color, description, enabled
Rule	id, name, condition, threshold, severity, enabled
Scenario	id, name, description, asset, expectedResults, tags

Para la demo puede usarse almacenamiento local tipado y datos semilla. En producción se recomienda PostgreSQL para metadatos, almacenamiento de objetos para imágenes/resultados pesados y políticas de retención, acceso y auditoría.

9. Contratos TypeScript de referencia

```
export type AnalysisMode = "detection" | "segmentation" | "patterns" | "combined";
export type AnalysisStatus = "queued" | "processing" | "completed" | "failed";

export interface BoundingBox {
  x: number; y: number; width: number; height: number; // 0..1
}
export interface Detection {
  id: string; label: string; classId: string;
  confidence: number; box: BoundingBox;
}
export interface PatternResult {
```

```
  id: string; type: string; label: string;
  confidence: number; evidence: string;
}
export interface VisionResult {
  analysisId: string;
  detections: Detection[];
  masks: SegmentationResult[];
  patterns: PatternResult[];
  durationMs: number;
}
export interface VisionEngine {
  analyze(input: AnalysisInput): Promise<VisionResult>;
}
```

10. Simulación funcional: escenarios demostrativos

Escenario	Objetos / patrones esperados	Visualización
Seguridad industrial	Persona, casco, chaleco; patrón EPP incompleto.	Cajas por objeto, máscara de persona y alerta revisable.
Almacén inteligente	Caja, pallet, montacargas; objeto en zona restringida.	Conteo por clase, regiones y alerta de zona.
Control de calidad	Producto, etiqueta, defecto superficial simulado.	Máscara de defecto, clase conforme/no conforme.
Retail	Persona, estante, producto; ocupación alta.	Conteo, mapa de regiones y KPI de afluencia.

Para que la demo sea reproducible, cada escenario debe tener resultados semilla asociados a la imagen de ejemplo. La misma entrada produce el mismo resultado, salvo que se habilite explícitamente un modo aleatorio de demostración. Las coordenadas deben estar calibradas para el recurso visual usado.

11. API prevista para conectar IA real

Método / ruta	Entrada	Salida
POST /api/v1/analyses	imagen o assetId, mode, threshold, scenarioId	analysisId, status
GET /api/v1/analyses/{id}	id	estado y métricas
GET /api/v1/analyses/{id}/results	id	detections, masks, patterns, alerts
GET /api/v1/analyses	filtros, paginación	historial paginado
PATCH /api/v1/alerts/{id}	status, note	alerta actualizada
GET /api/v1/dashboard	rango de fechas	KPIs y series agregadas

Para análisis largos, el backend puede responder con un ID y notificar progreso mediante WebSocket o Server-Sent Events. La demo puede simular etapas de progreso en el frontend, sin afirmar que se ejecuta inferencia en servidor.

12. Seguridad, privacidad y calidad

- Validar tipo MIME, extensión y tamaño; no confiar solo en el nombre del archivo.
- Evitar exponer imágenes privadas en URLs públicas; aplicar permisos por rol y enlaces temporales.
- Definir consentimiento, finalidad, retención y borrado de imágenes, especialmente si aparecen personas.
- Registrar auditoría de cambios de reglas, revisión de alertas y exportaciones.
- Sanitizar nombres y metadatos; limitar tamaño y frecuencia de solicitudes.
- Separar datos demo de datos reales y etiquetar claramente cada origen.
- Probar con iluminación, ángulos, oclusiones y diversidad de escenarios; documentar límites y sesgos.

13. Plan de desarrollo por etapas

Etapa	Entregables	Criterio de salida
1. Base del proyecto	React + TS, layout, rutas, tema, navegación y componentes.	Build correcto y rutas navegables.
2. Dominio y datos demo	Tipos, repositorio, escenarios, reglas y datos semilla.	Datos coherentes y tipados.
3. Dashboard	KPI, gráficos, actividad y alertas.	Indicadores derivados del repositorio.
4. Centro de análisis	Carga/selección de imagen, visor, progreso y overlays.	Análisis demo ejecutable de inicio a fin.
5. Segmentación y patrones	Máscaras, leyendas, reglas y paneles de detalle.	Resultados filtrables y explicables.
6. Historial y reportes	Búsqueda, filtros, detalle y CSV.	Persistencia local y exportación válida.
7. Calidad y presentación	Responsive, accesibilidad, pruebas, animaciones y demo guiada.	Checklist aprobado y sin errores críticos.
8. IA real (fase 2)	API Python, modelo validado, almacenamiento y monitoreo.	Métricas sobre dataset etiquetado y despliegue controlado.

14. Pruebas y criterios de aceptación

- Se puede seleccionar un escenario y ejecutar un análisis sin errores.
- El visor representa correctamente cajas y máscaras en la imagen, incluso al cambiar el tamaño del panel.
- Los filtros por clase, confianza, fecha y estado actualizan los resultados.
- El historial conserva los análisis después de recargar (si se implementa persistencia local).
- Las alertas pueden marcarse como revisadas y el cambio se refleja en dashboard e historial.
- Los KPI se calculan desde los análisis almacenados y no son números decorativos desconectados.
- Se puede exportar un CSV con identificador, fecha, escenario, clase, confianza y estado.
- El sistema diferencia de forma inequívoca DEMO y REAL.
- Los errores de carga y análisis se muestran con mensajes accionables.
- Las animaciones no bloquean la interacción y respetan la preferencia de movimiento reducido.

15. Guion de demostración para empresa

9. Presentar dashboard: volumen de análisis, detecciones y alertas recientes.
10. Entrar al escenario Seguridad industrial y explicar la imagen de entrada.
11. Ejecutar análisis combinado y observar progreso animado.
12. Explicar la diferencia entre bounding box (detección) y máscara de píxeles (segmentación).
13. Mostrar patrón detectado, nivel de confianza y evidencia visual.
14. Revisar una alerta, cambiar su estado y demostrar trazabilidad.
15. Consultar historial, filtrar por escenario y exportar el reporte.
16. Cerrar explicando que el adaptador simulado puede reemplazarse por un modelo real mediante el contrato VisionEngine.

16. Recomendación tecnológica

Área	Primera versión	Evolución productiva
Frontend	React + TypeScript + Vite; Tailwind/shadcn; Recharts; Lucide.	PWA, pruebas E2E y telemetría.
Motor demo	DemoVisionAdapter con escenarios deterministas.	Adaptador API sin cambiar la UI.
Backend	Opcional en demo; local repository.	Python FastAPI, workers y colas.
Visión	Resultados predefinidos para enseñar el flujo.	OpenCV + detector/segmentador validado.
Datos	LocalStorage/IndexedDB para prototipo.	PostgreSQL + almacenamiento de objetos.

Tiempo real	Progreso simulado y notificaciones in-app.	WebSocket/SSE y eventos persistidos.

17. Glosario

Término	Definición breve
Bounding box	Rectángulo que delimita la ubicación aproximada de un objeto.
Máscara	Mapa de píxeles que identifica la región de un objeto o clase.
IoU	Medida de solapamiento entre una predicción y la referencia real.
Confianza	Puntuación estimada asociada a una predicción.
NMS	Técnica para eliminar cajas duplicadas o muy solapadas.
mAP	Métrica agregada para evaluar detección a distintos umbrales.
Embedding	Representación numérica compacta de características visuales.
Inferencia	Ejecución de un modelo entrenado sobre datos nuevos.

18. Resultado esperado

Al finalizar la primera versión se tendrá una aplicación empresarial navegable y demostrable, con dashboard conectado a datos de prueba, centro de análisis visual, detección con cajas, segmentación con máscaras, reconocimiento de patrones, alertas, historial, exportación y animaciones profesionales. El código quedará organizado por módulos y capas, de modo que la simulación pueda evolucionar a un servicio real de visión computacional sin reconstruir la interfaz.

FIN DEL DOCUMENTO